

Computer Architecture - Homework #2

Ziyi Gu
999019607

Teng Lei
999019730

May 6, 2026

1 System Information

Output of `papi_avail -a`:

```
PAPI_L2_ICH 0x8000004a No Level 2 instruction cache hits
PAPI_L1_ICA 0x8000004c No Level 1 instruction cache accesses
PAPI_L2_ICA 0x8000004d No Level 2 instruction cache accesses
PAPI_L1_ICR 0x8000004f No Level 1 instruction cache reads
PAPI_L1_TCH 0x80000055 Yes Level 1 total cache hits
PAPI_L2_TCH 0x80000056 Yes Level 2 total cache hits
PAPI_L1_TCA 0x80000058 Yes Level 1 total cache accesses
PAPI_L2_TCA 0x80000059 No Level 2 total cache accesses
PAPI_FML_INS 0x80000061 No Floating point multiply instructions (Also includes multiply-add instructions)
PAPI_FAD_INS 0x80000062 No Floating point add instructions (Also includes subtract and multiply-add instructions)
PAPI_FDV_INS 0x80000063 No Floating point divide instructions (Counts both divide and square root instructions)
PAPI_FSQ_INS 0x80000064 No Floating point square root instructions (Counts both divide and square root instructions)
PAPI_FP_OPS 0x80000066 No Floating point operations
PAPI_SP_OPS 0x80000067 No Floating point operations; optimized to count scaled single precision vector operations
PAPI_DP_OPS 0x80000068 No Floating point operations; optimized to count scaled double precision vector operations
-----
Of 41 available events, 8 are derived.
```

2 PAPI Initialization and Initial Tests

1. Why must PAPI be initialized before starting measurements?

PAPI must be initialized before measurements because the library needs to set up its internal data structures and connect to the hardware counters. Without initialization, PAPI does not know which counters are available or how to access them.

2. What happens if counters are not stopped correctly?

If counters are not stopped correctly, the measured values may be wrong or incomplete. The program may also keep counting beyond the region we wanted to measure, so the result would include extra work.

3. Why is it important to isolate the measured region?

It is important to isolate the measured region because we only want to measure the core computation. If we include memory allocation, initialization, or printing, the counter values will not describe the real behavior of the algorithm.

4. How many counters are available in your system?

The machine reports 5 hardware counters.

5. What is the meaning of the constant `PAPI_DOM_ALL`?

`PAPI_DOM_ALL` means that PAPI counts events from all execution domains available to the process, including both user mode and kernel mode. In this assignment, it was used so the counters include the full execution behavior of the measured region.

3 Sequential Cache Behavior

Size	L1-I Misses	L1-D Misses	L2 Misses	LLC Misses	# instructions	# cycles
1000	283	111	36	—	11696	55122
10000	275	100	24	—	101696	423203
100000	618	459	246	—	1010069	4164436
1000000	2016	1019	629	—	10029256	41245563
10000000	9057	13297	3670	—	100471933	411781051
100000000	163654	56849	4670	—	1002186040	4107984496
1000000000	1174074	652571	105655	—	10025558941	41106211521

Table 1: Cache behavior using one thread

3.1 Questions and Analysis

1. **How do cache misses evolve with input size? Create a graph showing this.**

In general, the measured cache misses increase as the input size grows. For small sizes, the values are low and somewhat noisy. For example, L1-D misses are 111 for size 10^3 and 100 for size 10^4 . For larger sizes, the increasing trend becomes clearer. L1-I misses grow from 283 at 10^3 to 1,174,074 at 10^9 , L1-D misses grow from 111 to 652,571, and L2 misses grow from 36 to 105,655.

The graph shows that the miss counts do not grow perfectly smoothly, but larger inputs generally lead to more observed cache misses.

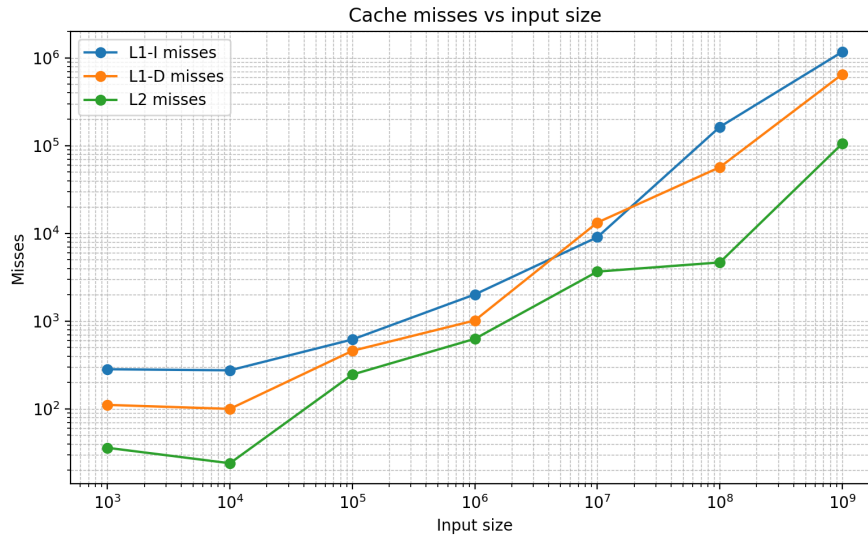


Figure 1: Cache misses vs. input size

2. **At which point do you observe significant increases in misses?**

The first significant increase appears around size 10^7 . At this point, L1-D misses increase from 1,019 at 10^6 to 13,297 at 10^7 , and L2 misses increase from 629 to 3,670. Another clear jump appears at 10^8 , where L1-I misses increase from 9,057 to 163,654. The largest overall increase happens at 10^9 , where L1-D misses reach 652,571 and L2 misses reach 105,655.

3. How does this relate to cache sizes?

At first, it may seem that the increase in cache misses for larger input sizes is caused by the program exceeding the cache capacity. However, in this benchmark the input size only controls the number of loop iterations. The code repeatedly updates a few local variables, mainly `tmp` and `i`, so the actual data footprint is very small and should easily fit in L1 cache. Therefore, the increase in misses is not mainly caused by the data becoming too large for the cache. After searching and asking AI, a more likely explanation is that larger sizes make the program run for much longer. During longer executions, the process is more likely to be interrupted by the operating system or affected by context switches and other background activity. After such interruptions, some cache lines may need to be reloaded, which can increase the observed L1 and L2 miss counts.

LLC misses could not be analyzed because the assigned machine did not expose a usable PAPI counter for them.

4. How does the number of executed instructions evolve with the input size. And the number of cycles? Create a graph to support your analysis.

Both the number of executed instructions and the number of cycles grow approximately linearly with the input size. This is expected because the benchmark loop performs a fixed amount of work in each iteration.

Therefore, the graph of instructions and cycles versus input size should appear almost linear on a log-log scale.

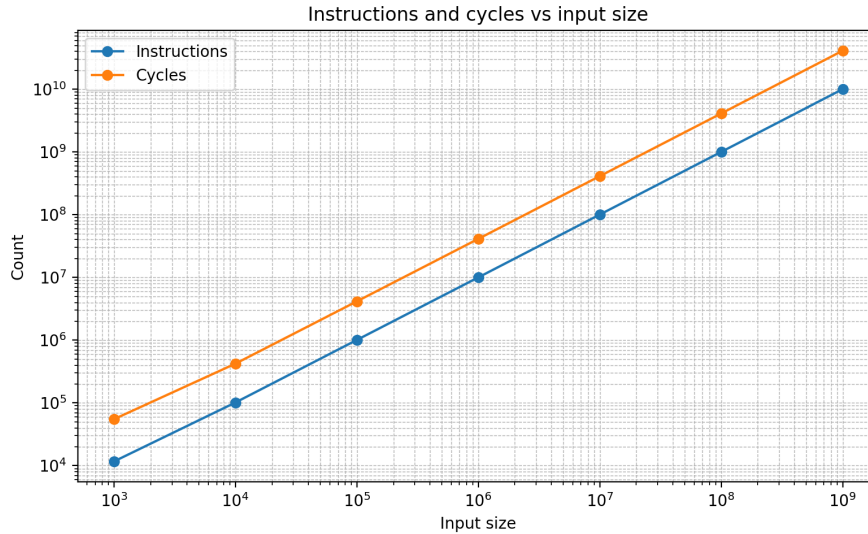


Figure 2: Instructions and cycles vs. input size

4 Parallel Cache Behavior

The original SMP cache-line events requested in the assignment were not available on the assigned machine. I used available replacement events: `PAPI_RES_STL`, `PAPI_STL_ICY`, and `PAPI_BR_PRC`. The program was executed with 1, 2, 4, 8, and 16 threads, and each thread ran the same task.

Size	L1-I Misses	L1-D Misses	L2 Misses	# instructions	# cycles	Resource Stalls	No-Issue Cycles	Branch Correct	Time (s)
1000	40	19	27	10189	44974	36995	2531	1031	0.000390607
10000	48	10	26	100195	414911	364308	6501	10031	0.000313080
100000	75	41	33	1000193	4111080	3700030	2607	100027	0.003301547
1000000	90	43	26	10000195	41009597	36983432	13376	1000025	0.013458237
10000000	228	238	89	100000259	410133082	369959444	76788	9999997	0.137406677
100000000	2508	1737	51	1000000853	4100471406	3680111611	984580	99999734	1.326229491
1000000000	12771	16401	492	10000007249	41016738731	36810267549	16009129	999996117	13.282110194

Table 2: Parallel cache behavior with 1 thread

Size	L1-I Misses	L1-D Misses	L2 Misses	# instructions	# cycles	Resource Stalls	No-Issue Cycles	Branch Correct	Time (s)
1000	81	30	58	20390	90990	74881	7655	2056	0.000862104
10000	78	27	60	200388	831684	740622	7058	20060	0.000985395
100000	90	60	62	2000378	8210606	7389619	13474	200053	0.001724931
1000000	155	109	69	20000404	82031263	73922815	28990	2000049	0.018127674
10000000	312	340	65	200000556	820475522	739307369	711486	19999970	0.138098514
100000000	2749	3298	232	2000002083	8204603600	7390373016	2339605	199999329	1.336010750
1000000000	18914	31725	1220	20000015873	82030085629	73609545723	25426157	1999993079	13.289833832

Table 3: Parallel cache behavior with 2 threads

Size	L1-I Misses	L1-D Misses	L2 Misses	# instructions	# cycles	Resource Stalls	No-Issue Cycles	Branch Correct	Time (s)
1000	157	70	121	40774	181653	149610	15465	4112	0.001242690
10000	156	92	138	400762	1656303	1480449	17059	40118	0.001448502
100000	238	91	131	4000796	16439668	14780087	17355	400112	0.004021185
1000000	394	240	132	40000836	164104001	147880747	132478	4000091	0.020294544
10000000	1023	959	229	400001158	1640643994	1479667119	406031	39999948	0.148921807
100000000	4973	7442	457	4000004329	16405086817	14759238998	5091454	399998804	1.346433381
1000000000	42410	89310	2218	40000035896	164128637427	146695760794	50519908	3999985781	13.360243549

Table 4: Parallel cache behavior with 4 threads

Size	L1-I Misses	L1-D Misses	L2 Misses	# instructions	# cycles	Resource Stalls	No-Issue Cycles	Branch Correct	Time (s)
1000	218	142	157	81537	368606	174455	32961	8244	0.002085213
10000	564	169	237	801534	3319028	2141702	20536	80243	0.003648260
100000	291	265	194	8001561	32878496	21386782	56182	800244	0.004851287
1000000	533	559	206	80001742	333246006	213736324	249863	8000213	0.020514447
10000000	3028	1754	469	800002303	3285490467	1794884451	7716351	79999818	0.152408176
100000000	12690	14809	1179	8000009421	32879846068	26842251877	13059798	799997513	1.352831527
1000000000	103293	196043	5810	80000085966	328184804434	274295170410	215210212	7999969238	13.372139220

Table 5: Parallel cache behavior with 8 threads

Size	L1-I Misses	L1-D Misses	L2 Misses	# instructions	# cycles	Resource Stalls	No-Issue Cycles	Branch Correct	Time (s)
1000	435	278	326	163035	816327	264972	67386	16509	0.012234183
10000	472	345	283	1603208	6757908	2660600	193099	160506	0.010675016
100000	541	875	388	16003197	65831616	30737957	300313	1600491	0.011784419
1000000	744	1293	374	160003653	656860884	267290038	634665	16000440	0.030775440
10000000	2848	4143	440	1600004917	6562441463	2785279158	3695525	159999858	0.146430914
100000000	16070	29201	1118	16000020410	65687180046	26485284100	52309871	1599993834	1.368626146
1000000000	175486	285138	6188	160000297613	656858277892	265011751979	665720840	15999903453	13.952013982

Table 6: Parallel cache behavior with 16 threads

4.1 Questions and Analysis

1. Does increasing the number of threads increase or decrease cache efficiency?

Increasing the number of threads decreases cache efficiency. However, similar to analysis in 3.1.3, the program does not use a large shared array. Each thread mostly updates local variables, so there is not much true shared-data cache traffic. Therefore, the increase in misses is mostly due to running more total work and having more active threads, rather than because the cache is heavily stressed by shared data.

2. Is there evidence of cache contention or bandwidth limitations?

There is only limited evidence of cache contention, and little evidence of a strong memory bandwidth limitation. Similar analysis as above, although for example L1-D misses and L2 misses increase which may suggest that having more active threads creates more pressure on the cache hierarchy. The measured increase in misses is not strong evidence of severe cache contention since the benchmark does not access a large shared data structure. It is more likely a combination of more total execution and some competition for shared cache resources. There is also no strong evidence of a memory bandwidth bottleneck. In this benchmark, the memory footprint is very small, so the program is unlikely to be limited by memory bandwidth.

3. How does cache behavior correlate with execution time?

Cache misses increase as the execution time becomes longer, but the correlation is not perfectly linear. The relationship is positive but weak, higher miss counts are associated with slightly longer execution time, but the increase in time is much smaller than the increase in cache misses. Also since the program has a very small data footprint, I think the stronger correlation is between execution time and the amount of total work. Moreover, the PAPI_BR_PRC counter also increases almost proportionally with the input size and thread count, since these are correctly predicted branches, they do not suggest that branch behavior is a major source of performance loss.

5 Vector Processing and Computing π

The events PAPI_VEC_SP and PAPI_VEC_DP were tested, but they were not available on the assigned machine, so the corresponding values are not reported. For vector addition, size 10^9 was not executed because it would require three arrays of 10^9 doubles, about 24 GB of memory.

5.1 Vector Addition

Threads	Size	Time (s)	L1-I Misses	L1-D Misses	L2 Misses	# instructions	# cycles
1	10^3	0.000379908	65	129	34	6258	11339
1	10^4	0.000240147	94	1117	76	60280	68632
1	10^5	0.001581369	99	15261	927	600458	769183
1	10^6	0.009510647	257	146155	11549	6002218	15085984
1	10^7	0.095532981	585	1582269	90646	60019826	138345881
1	10^8	0.897026418	6346	14284687	902659	600195806	1370172736
2	10^3	0.000790906	91	156	115	25147	39734
2	10^4	0.000929399	122	1116	161	188935	233474
2	10^5	0.001558079	238	15414	9880	1366415	1869426
2	10^6	0.008230322	303	148785	14421	8102483	14848286
2	10^7	0.049484250	1880	1613605	99416	62120103	137064702
2	10^8	0.460777812	5784	14443166	865054	602296050	1234609617
4	10^3	0.001302917	150	259	129	34127	65117
4	10^4	0.001251170	156	1198	215	177461	296559
4	10^5	0.001906545	216	15326	1293	774990	1452919
4	10^6	0.005818521	485	144130	9951	9107518	16267773
4	10^7	0.035269579	1107	1606559	86569	66320623	182039451
4	10^8	0.299388644	5484	14345967	799188	606496666	1618929973
8	10^3	0.002274112	292	406	312	51810	120105
8	10^4	0.001955119	344	1283	375	224333	377937
8	10^5	0.002510415	360	16053	2967	945810	1508222
8	10^6	0.007277568	1146	150134	28305	14855393	25928116
8	10^7	0.028952017	1877	1565704	246488	73064611	270072709
8	10^8	0.229260495	15714	14439129	2498055	614897837	2194077522
16	10^3	0.008458378	544	830	525	20832139	28442846
16	10^4	0.008147765	923	1659	1088	19643262	27078390
16	10^5	0.014254655	591	30289	2238	21924120	30076710
16	10^6	0.014609196	960	144251	29799	37506236	64840356
16	10^7	0.025534544	3459	1548426	315092	91523879	306793141
16	10^8	0.219199555	10995	14353115	6630137	631700378	2996526759

Table 7: Vector addition PAPI measurements

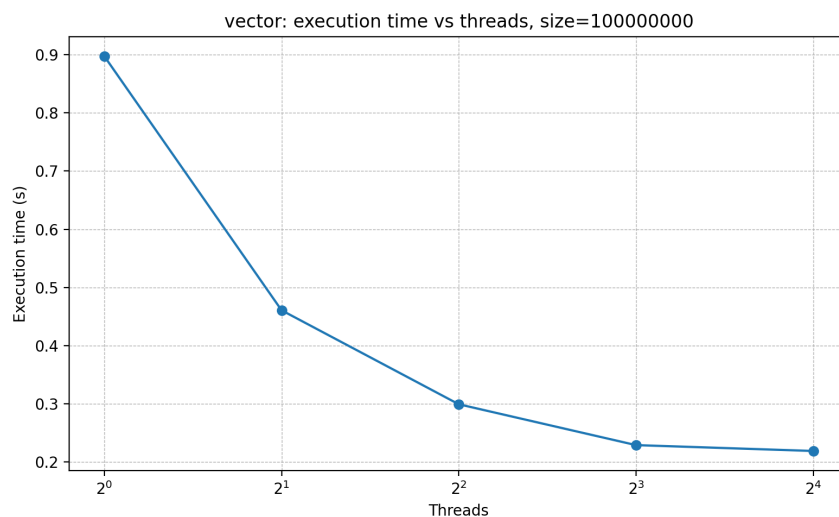


Figure 3: Vector addition execution time vs. threads for size 10^8

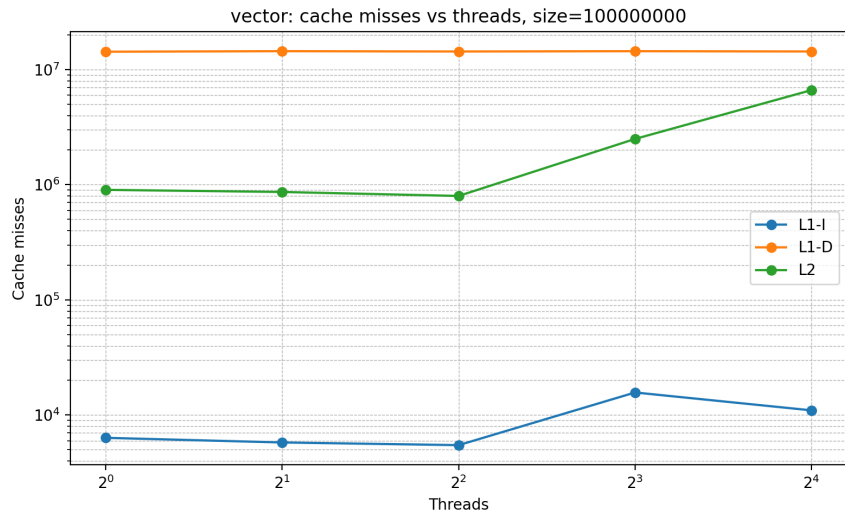


Figure 4: Vector addition cache misses vs. threads for size 10^8

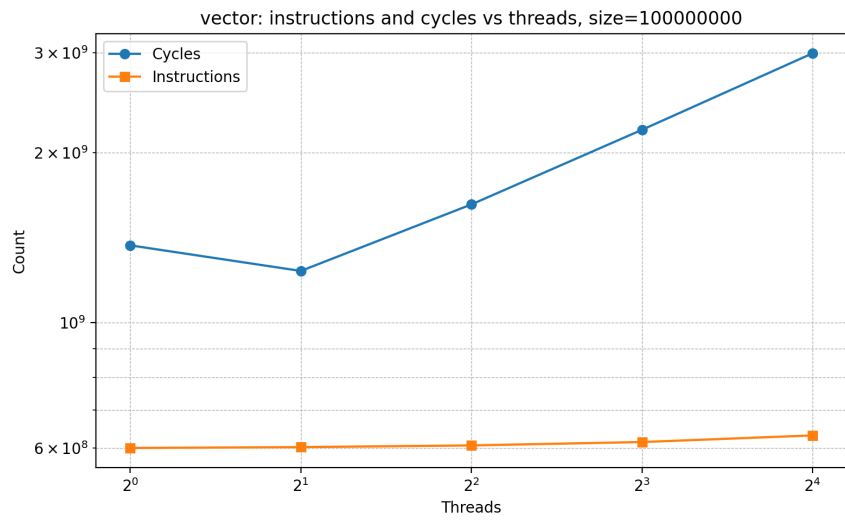


Figure 5: Vector addition additional counters vs. threads for size 10^8

5.2 Computing π

Threads	Size	Time (s)	L1-I Misses	L1-D Misses	L2 Misses	# instructions	# cycles
1	10^3	0.000373255	42	22	28	12263	15651
1	10^4	0.000396070	48	19	29	120269	111070
1	10^5	0.001187301	85	19	26	1200265	1051721
1	10^6	0.006074069	95	46	33	12000267	10453847
1	10^7	0.038103760	128	94	37	120000289	115090122
1	10^8	0.361157580	727	407	173	1200000551	1092586146
1	10^9	3.452171708	2063	6432	278	12000002777	10649140301
2	10^3	0.000878559	96	47	70	41432	53461
2	10^4	0.000804515	98	49	64	202655	202227
2	10^5	0.001084136	100	87	74	1825545	1712109
2	10^6	0.004439431	143	89	63	14100540	12639827
2	10^7	0.026148462	227	142	135	122100570	116653130
2	10^8	0.184191758	281	472	249	1202100752	1070365936
2	10^9	1.745098587	2659	4018	469	12002102624	10717909756
4	10^3	0.001211259	201	95	110	25278	52582
4	10^4	0.001179553	162	91	105	184288	248276
4	10^5	0.001385340	132	158	96	1205274	1071463
4	10^6	0.005147669	203	188	201	16209252	20873554
4	10^7	0.021928280	319	242	227	126301150	160428046
4	10^8	0.165281171	831	614	471	1206301513	1501818962
4	10^9	0.909097725	3832	4293	631	12006303362	10834745096
8	10^3	0.002242028	243	197	249	70818	127553
8	10^4	0.002143978	293	228	151	301567	465821
8	10^5	0.002877364	265	189	174	2518131	3221984
8	10^6	0.005232530	399	225	248	19482993	28158691
8	10^7	0.014079660	774	507	398	132620544	177671699
8	10^8	0.091561853	1306	1123	485	1212832602	1610017289
8	10^9	0.719836598	3528	8195	1046	12014705464	14583029558
16	10^3	0.009756495	607	461	370	23650287	31421584
16	10^4	0.007909747	741	701	358	264943	486259
16	10^5	0.008811527	840	493	326	15002045	21075826
16	10^6	0.008372068	672	708	691	26562718	41639686
16	10^7	0.022269181	1029	940	740	151504478	214461469
16	10^8	0.060157154	1780	2007	965	1231504969	2001735387
16	10^9	0.491217767	7434	13152	1416	12031513139	19710283409

Table 8: Computing π PAPI measurements

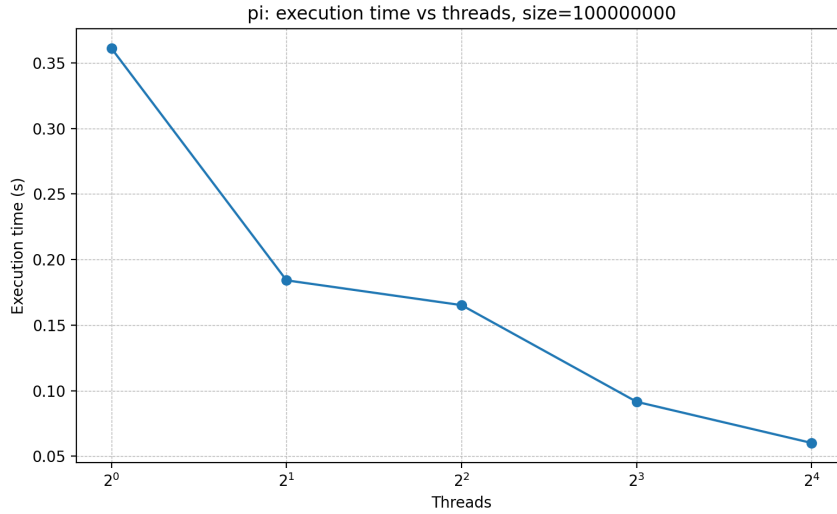


Figure 6: Computing π execution time vs. threads for size 10^8

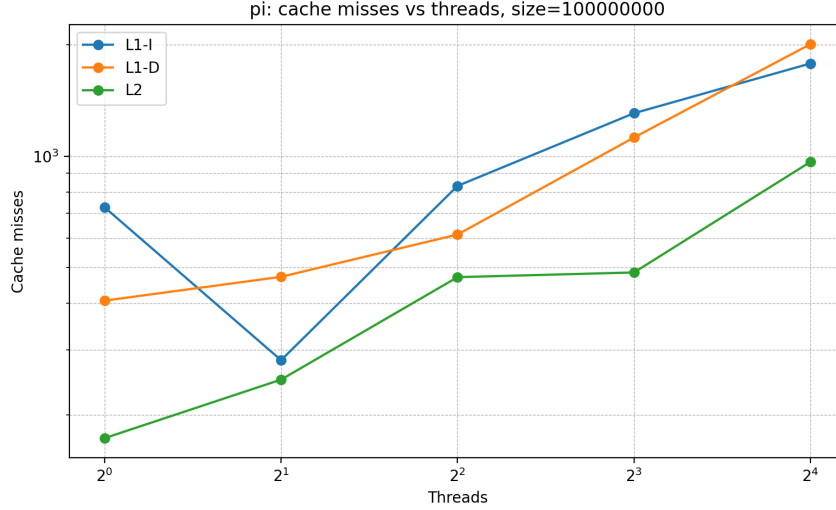


Figure 7: Computing π cache misses vs. threads for size 10^8

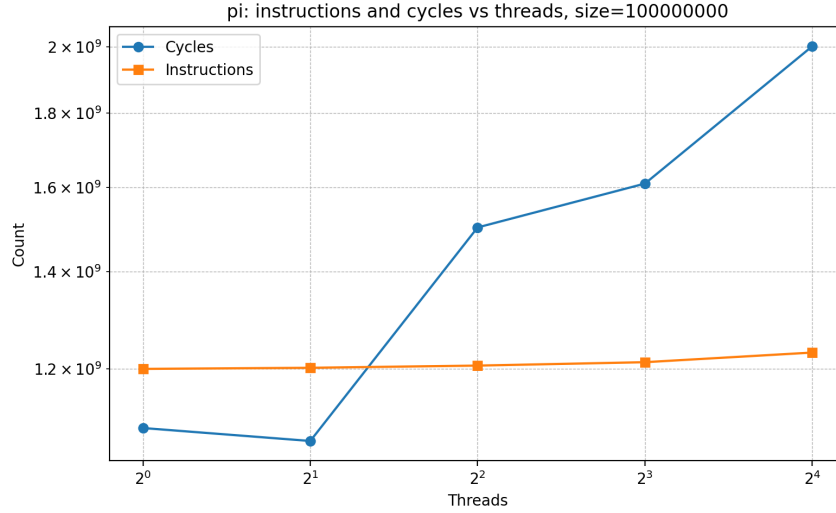


Figure 8: Computing π additional counters vs. threads for size 10^8

5.3 Questions and Analysis

- **Can performance degradation be explained by cache behavior?**

For vector addition, cache behavior can explain part of the performance limit. When the number of threads increases, the program becomes faster, but the speedup is not perfect. For size 10^8 , the time goes from 0.897 s with 1 thread to 0.219 s with 16 threads. At the same time, L2 misses increase from 902,659 to 6,630,137. This means that the cache is under more pressure when more threads are used.

For the π program, cache misses do not explain the performance very much. The number of cache misses is much smaller than in vector addition. For size 10^8 , L2 misses only increase from 173 to 965, while the time improves from 0.361 s to 0.060 s. So for this program, cache behavior is not the main limitation.

- **Which cache level has the strongest impact on performance?**

For vector addition, the L2 cache seems to have the strongest impact. L1-D misses are already large because the program reads and writes big arrays. But the biggest change is in L2 misses. For size 10^8 , L2 misses increase from 799,188 with 4 threads to 2,498,055 with 8 threads, and then to 6,630,137 with 16 threads. This happens while the improvement in execution time becomes smaller.

For the π program, no cache level has a very strong impact. The program mostly works with a few scalar variables, so it does not create many cache misses.

- **Is the cache coherence affecting the performance?**

There is no strong evidence that cache coherence is a big problem here. The cache-coherence events from the assignment were not available on this machine, so I could not measure them directly.

Looking at the programs, vector addition gives different array elements to different threads, and the π program uses a reduction. So the threads are not constantly writing to the same memory location in the main loop. Because of this, cache coherence may add some overhead, but it does not seem to be the main reason for the performance behavior.

- **Is the application memory-bound or compute-bound?**

Vector addition is more memory-bound. In each iteration, it only performs one addition, but it has to read two arrays and write one array. So the program depends on moving data through memory and cache.

The π program is more compute-bound. Each iteration does arithmetic using a few local variables, and it does not read or write large arrays. Its cache miss counts are much smaller, so memory access is not the main bottleneck.